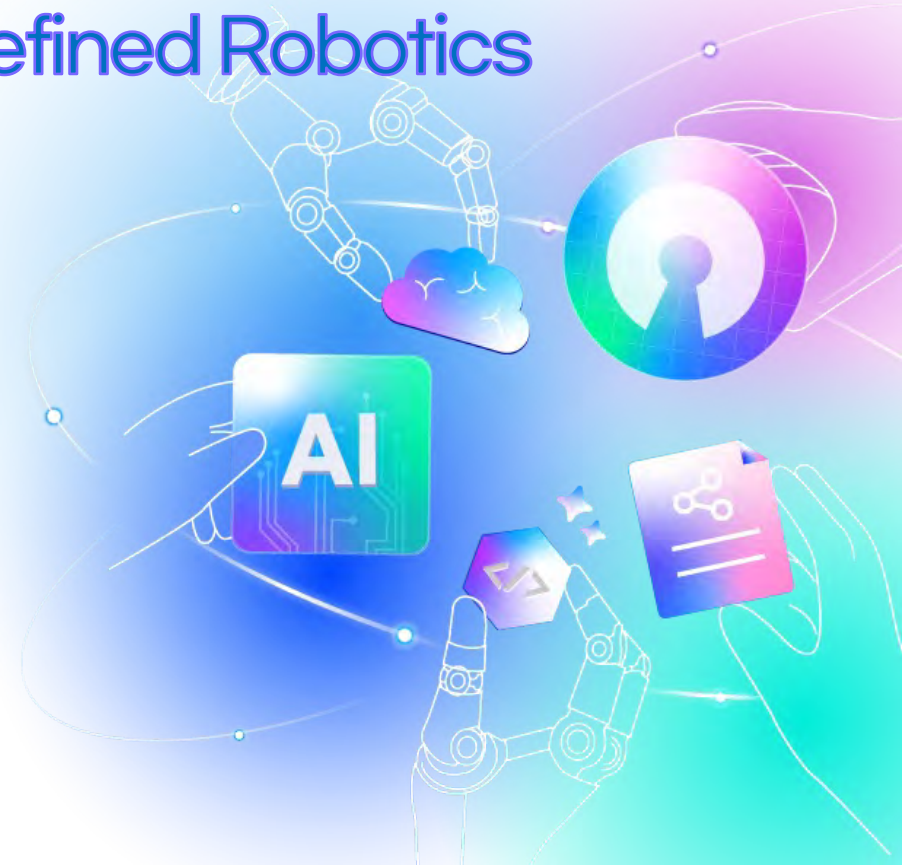


ARM·x86를 넘어: RISC-V로 만드는 개방형 컴퓨팅 플랫폼 Vela

RISC-V System Software for Software-Defined Robotics

안신영
ETRI



CONTENTS

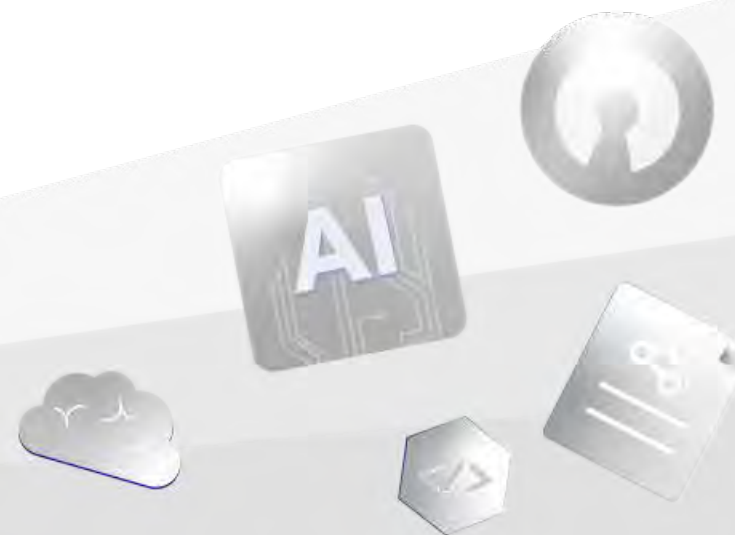
01 연구 과제

02 RISC-V 동향

03 RISC-V Vela Project

04 Release 및 라이선스 현황

05 결론





01

연구 과제

01

연구과제 - 개요

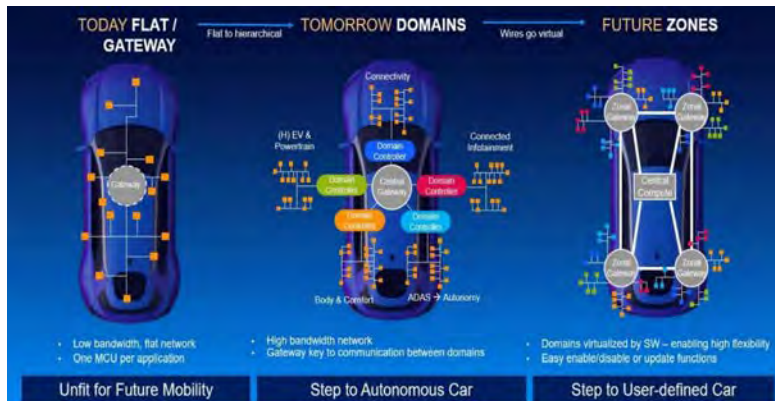
과제명	SDR의 개방형 생태계 구축을 위한 RISC-V 기반 시스템 소프트웨어 기술개발		
과제분류	SW·AI - 시스템 소프트웨어 - 운영체제 (혁신도약형, 공개SW)		
기술준비도 (TRL)	시작 시점 (3단계) → 종료 시점 (6단계) 1단계: 기초이론 / 실험 (T1) 2단계: 실용목적 아이디어 특허 등 개념정립 3단계: 연구실 규모의 기본성능 평가 (T2) 4단계: 연구실 규모의 부품/시스템: 성능평가 5단계: 부품/시스템 시작품 제작 (T3) 6단계: 시작품 성능평가 7단계: 시제품 선의성평가 8단계: 시제품 인증 및 표준화 9단계: 사업화 (T4, T5)		주관 및 참여기관
	 서울대 고려대 티라로보틱스 연세대 충북대 FALINUX Future Advanced Linux Georgia Tech		총괄책임자 안 신 영
총 연구기간	2024.7.1 ~ 2027.12.31 (4년, 42개월)		

01

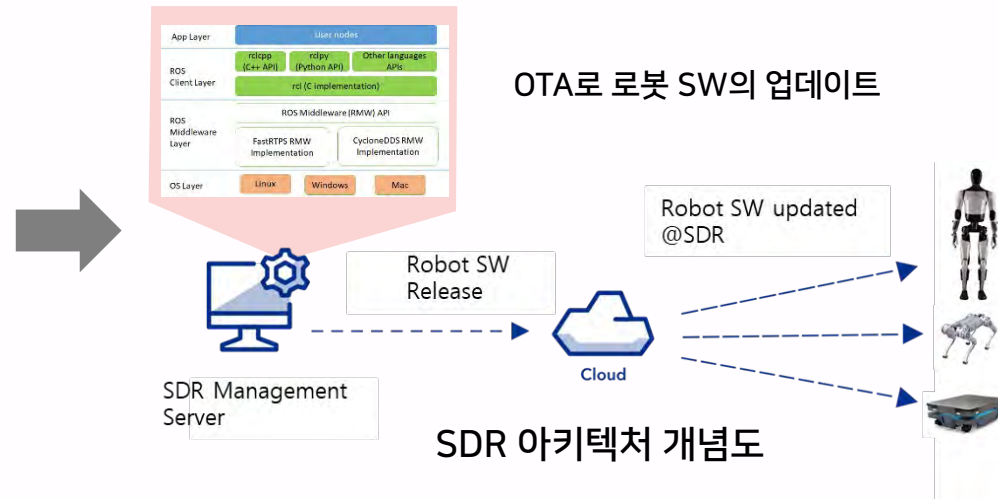
연구과제 - 목적

SDR(Software-Defined Robotics) : 이동성을 기반으로 다양한 지능작업을 제공하며,
소프트웨어 갱신, 교체에 의해 새로운 기능을 추가하기 용이한 지능형 서비스 로봇 기술

로봇산업에서 가장 취약한 가치사슬 고리(후방산업, 부품/모듈/SW)에서 경쟁력을 갖출 수 있는 방안 필요
→ SDV와 유사하게 SDR로 로봇개발 패러다임 전환 필요



Tesla SDV 아키텍처 개념도



로봇 후방산업으로서 SDR 경쟁력 제고 → SDR 개방형 생태계 구축 필요

01

연구과제 - SDR의 개방형 생태계 구축

온디바이스 SDR 컴퓨팅 플랫폼

<AS-IS> 폐쇄형 로봇 컴퓨팅 플랫폼

HW 유연성
부족

기술종속
(벤더락인)

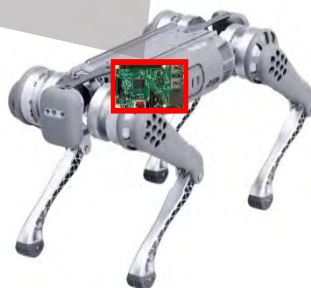
높은 로열티

ARM/X86기반 시스템 SW

NVIDIA, Qualcomm, Intel, ...
(ARM/X86+GPU/NPU)



자율이동로봇(AMR)



4족 보행로봇

<TO-BE> 개방형 SDR 컴퓨팅 플랫폼

자유로운 HW
최적화가능

기술종속
(벤더락인) 극복

로열티-프리

RISC-V 기반 시스템 SW

RISC-V HW



협동로봇(코봇)



휴머노이드

(폐쇄형) 외산 시스템 → (개방형) 로열티-프리 RISC-V기반 컴퓨팅 플랫폼

01

연구과제 - 연구목표

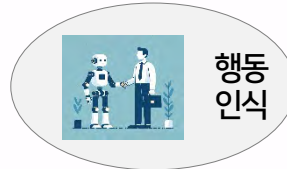
개방형 SDR 소프트웨어 생태계 강화를 위하여 **RISC-V 하드웨어 확장 기술** 연계
저전력, 고효율, 고신뢰, 고가용 실행환경 지원 **개방형 시스템 소프트웨어 기술 개발**



객체
인식



대화
이해



행동
인식

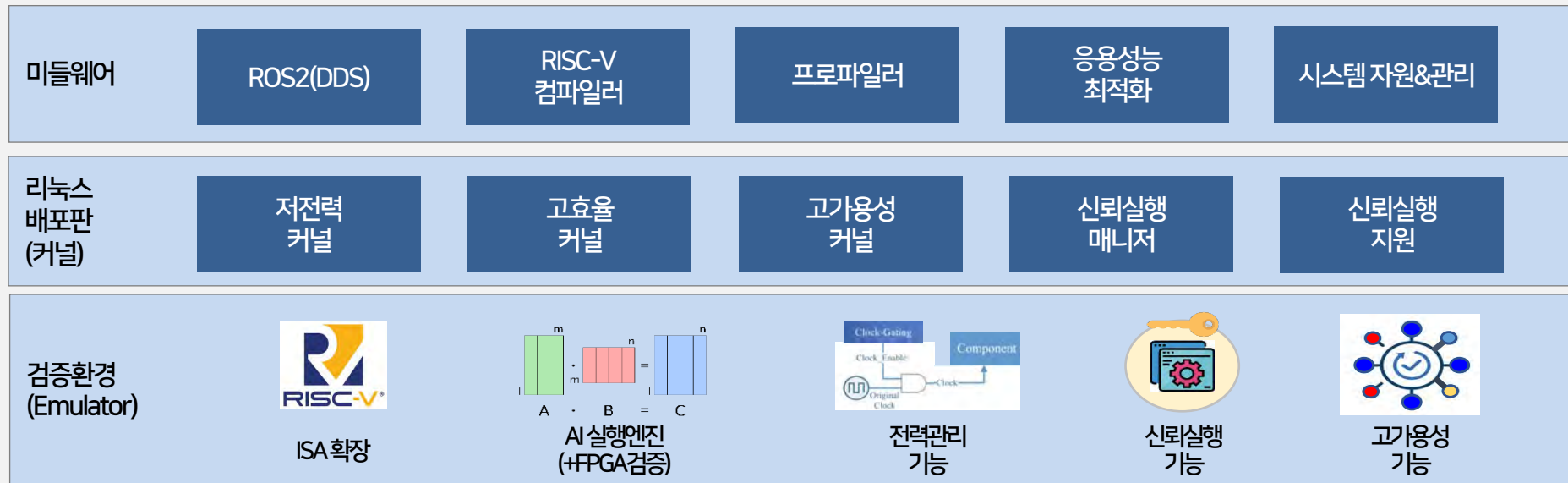


로봇
제어



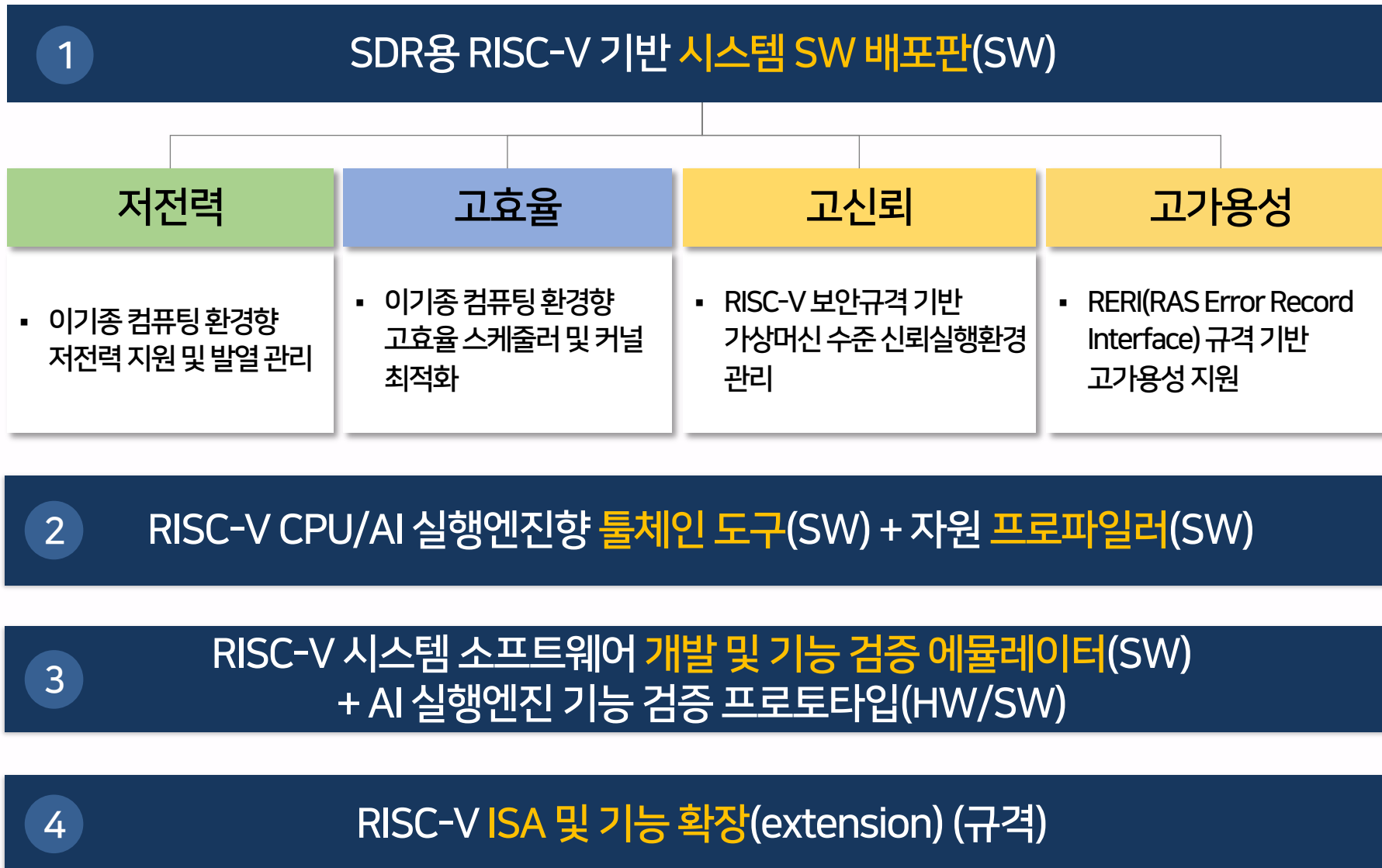
신뢰
응용

SDR용 RISC-V 기반 시스템 SW



01

연구과제 - 연구결과





02

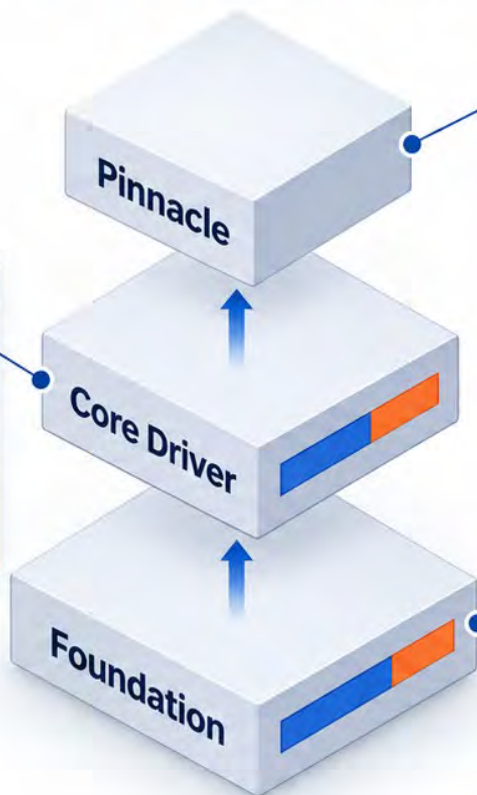
RISC-V 동향

RISC-V 시장 동향 - 2026년 3대 변곡점 과 2031년 전망

2026년, RISC-V의 변곡점

2. 빅테크의 수직 내재화 하이퍼스케일러의 M&A 가속

범용 서버 스타트업 모델의 한계 확인.
Meta의 Rivos 인수 및 Qualcomm의 Ventana 인수를 통해, 빅테크 기업들이
자사 LLM과 AI 추론 서비스에 특화된
칩 설계를 직접 내재화.



3. NVIDIA/GPU 생태계 편입 제3의 공식 호스트 CPU 격상

CUDA의 RISC-V 공식 이식, SiFive의 최초 NVLink Fusion 파트너 합류. x86과 ARM에
이어 데이터센터 GPU와 완벽히 결합하는
호스트 프로세서로 지위 상승.

1. 서버 플랫폼 표준 완성 Server Platform Spec 1.0 기준

RVA23 명령어셋 기반 UEFI-ACPI 6.6
도입 완료. 단일 OS 이미지가 벤더 다른
다기종 서버에서 구동되는 생태계 확립.
하드웨어 파편화를 방지하고 성능
하한선을 상향.

2031년, RISC-V 전망



33.7%

글로벌 실리콘 침투율
(10년간 10배 이상 폭발적 성장)

\$3,180억

2031년 RISC-V SoC 전체 시장 규모



359억 개

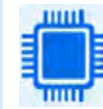
2031년 누적 SoC 출하 전망



02 RISC-V SoC 발전 동향

2021	2022	2023	2024	2025	2026
Linux 시작 D1, FU740	SBC대중화 JH7110	OoO·Edge TH1520/SG2042	RVV 1.0 확산 SpacemiT K1	성능고도화 P550/SG2044	AI+DataCenter K3/BigSky
					
● 1~4 코어 ● 초기 Linux 개발보드	● 4코어@1.5GHz ● VisionFive2	● 4xC910, 4TOPS ● 64xC920 서버 ● LicheePi 4A	● 8xX60, RVV 1.0, 2 TOPS, RVA22 ● Banana Pi BPI-F3	● 4xP550, 20TOPS SoC ● 64xC920, 2.6GHz 서버	● 8xX100, RVA23, RVV1.0, 60TOPS, ● 32xP870-D, 2GHz, RVA23 서버
RISC-V 실제로 만져볼 수 있는 시작점	저가 일반 사용자를 위한 SBC 생태계 확대	OoO 코어와 64 코어 서버 등장	RVV1.0기반 AI와 벡터처리 플랫폼 확산	서버급 RISC-V가 HPC/AI 플랫폼으로 진화	AI와 데이터센터 지향 RISC-V 플랫폼 본격화

핵심 추세



Linux 시작

단순코어 → OoO 고성능코어
→ 6-wide DC CPU



표준화

RVV초기구현 → RVV 1.0 → RVA23 확산



시장 확장

저가 Linux SBC → Edge AI SoC → Server/DataCenter



가격변화

개발보드 진입가격은 하락,
고성능 서버 플랫폼은 고급화



코어 수

1 → 4 → 8 → 32 / 64+



AI 연산

없음 → 4 TOPS → 20+ TOPS
→ 60 TOPS



메모리·I/O

DDR3/4 중심 → LPDDR5·DDR5,
PCIe Gen5



대표 폼팩터

SBC → Edge AI → Server

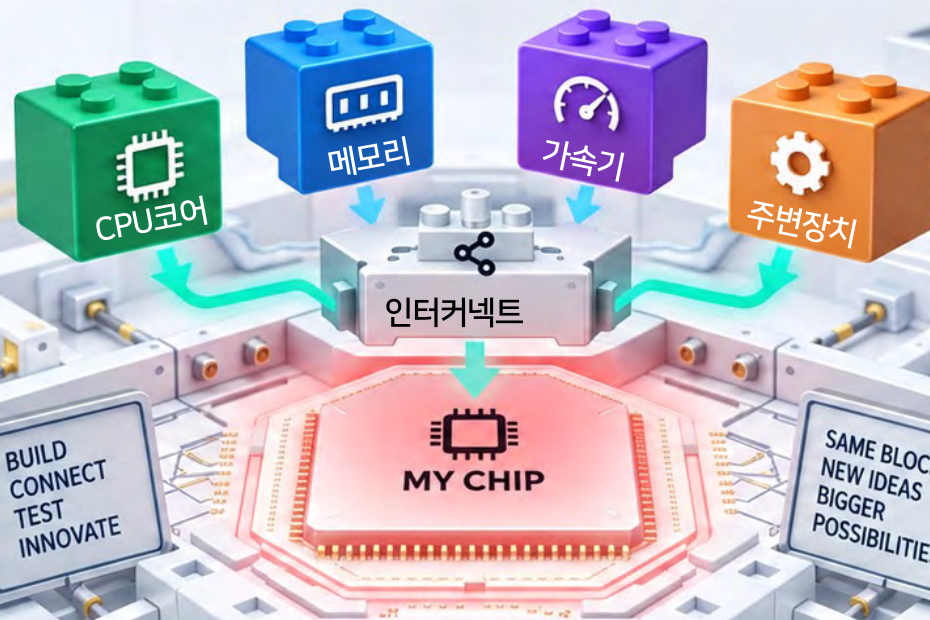
02

RISC-V 연구 동향 - Chipyard

Chipyard - 원하는 칩을 빠르게 설계·시험하는 오픈소스 플랫폼

쉽게 말하면

Chipyard는 칩 설계를
처음부터 전부 새로
그리는 대신, 검증된
블록을 골라 조립해 보는
설계 작업장



빠른 개발

처음부터 새로 만들지 않아
설계속도 높음



연구와 실험에 적합

새로운 구조 비교 및
시험하기 쉬움



1

블록을 고른다



프로세서, 메모리, 가속기
등 필요한 구성요소 선택

2

연결하고 설계한다



정해진 규칙으로 블록을
연어 하나의 칩 구조 생성

3

미리 시험한다



실제 제작전에
시뮬레이션과 검증으로
문제 줄임



RISC-V 기술 연구 동향 - Chisel

Chisel - 더 나은 프로세서를 만드는 Chipyard의 HW 설계 언어

하드웨어를 더 쉽고, 더 유연하고, 더 강력하게 설계할 수 있도록 만들어진 오픈소스 하드웨어 기술 언어

Chisel이란?

Chisel은 Scala 프로그래밍 언어를 기반으로 한 하드웨어 설계 언어(HDL)입니다. 복잡한 하드웨어를 텍스트로 쉽고 체계적으로 작성하고, 자동으로 Verilog 같은 하드웨어 기술 언어로 변환해 줍니다.



어떻게 동작하나요?

1. Chisel로 설계 작성

```
class Adder extends Module {  
  val io = IO(new Bundle {  
    val a = Input(UInt(32.W))  
    val b = Input(UInt(32.W))  
    val sum = Output(UInt(32.W))  
  })  
  
  io.sum := io.a + io.b  
}
```

친숙한 프로그래밍 방식으로
하드웨어 동작을 기술

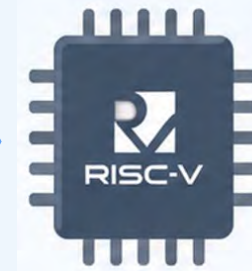
2. 자동변환

FIRRTL
(중간표현)

Velilog
(하드웨어기술언어)

컴파일러가 자동으로
하드웨어 언어로 변환

3. 하드웨어로 구현



실제 하드웨어에 탑재
(검증, FPGA, ASIC 등)

Chisel의 장점

-  **프로그래밍처럼 쉽다**
익숙한 Scala 문법을 사용해
복잡한 HW도 간결하게 작성
-  **높은 생산성**
재사용 가능 모듈과 라이브러리로
빠르게 설계
-  **높은 유연성, 확장성**
다양한 프로세서와 가속기를
파라미터로 쉽게 생성
-  **오픈소스**
전 세계 연구자가 함께 사용하고
발전시키는 생태계



03

RISCV-Vela Project

03 RISC-V-Vela Project

RISC-V System Software for Software-Defined Robotics



<https://github.com/riscv-vela>

The banner is titled 'Software Defined Robotics' and describes 'Intelligent, adaptable robots powered by open, software-defined technology.' It features four icons: a cloud with a gear (Updatable), a network of nodes (Intelligent), puzzle pieces (Modular), and a shield with a checkmark (Secure). The central image shows three robots: a small head-on-wheeled unit, a large industrial arm, and a humanoid robot. Below them is a circuit board with a glowing blue square labeled 'RISC-V Computing Platform'. To the right, a box titled 'RISC-V Computing Platform' states 'Open. Efficient. Scalable. Built for the future of robotics.' On the far right, a vertical list of four features is shown: 'Low Power' (leaf and battery icon), 'High Performance' (speedometer icon), 'High Reliability' (shield with padlock and TEE icon), and 'High Availability' (shield with heartbeat and gear icon).



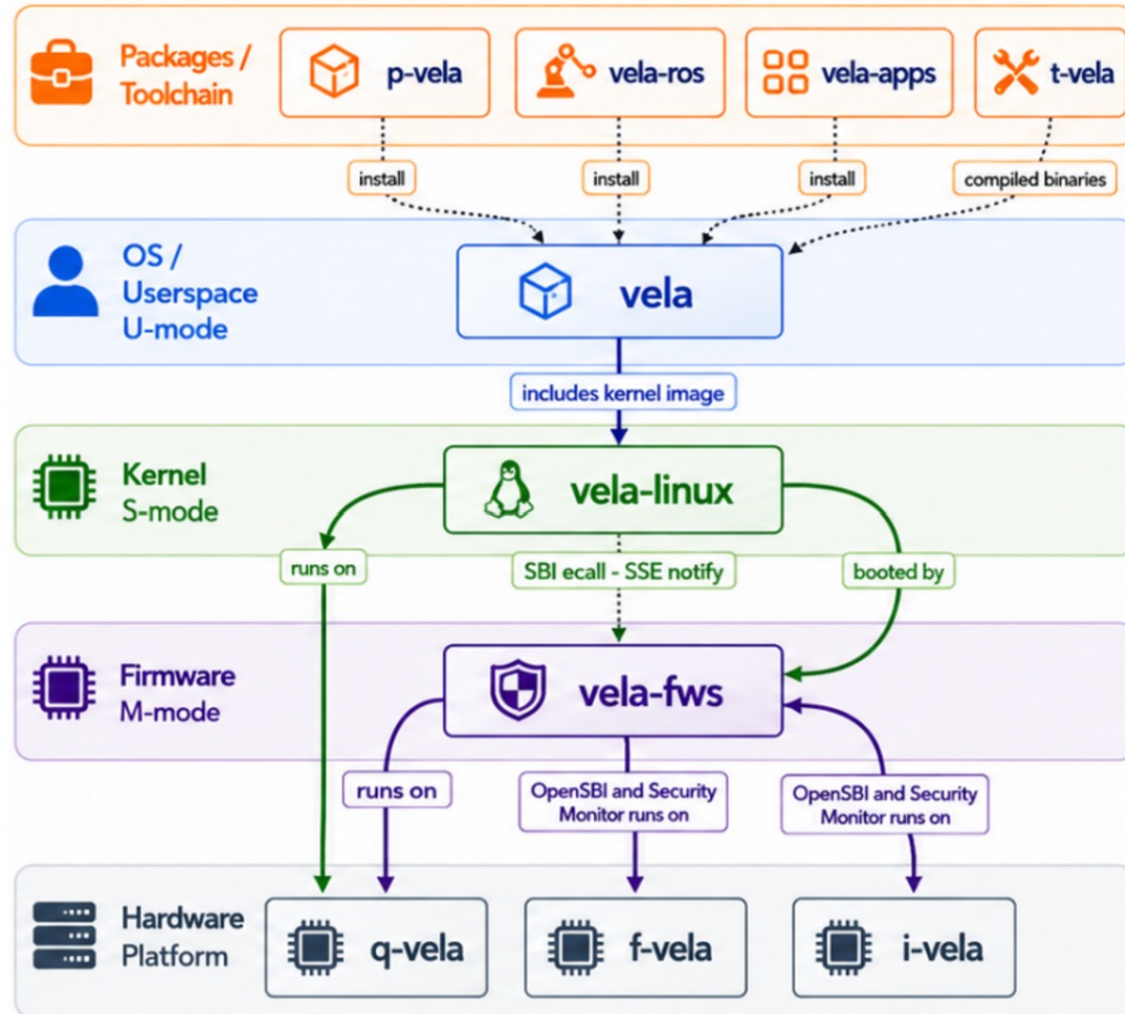
RISC-V Vela Project

10개의 저장소(Repositories) = 8 public + 2 private('26.12 공개 예정)

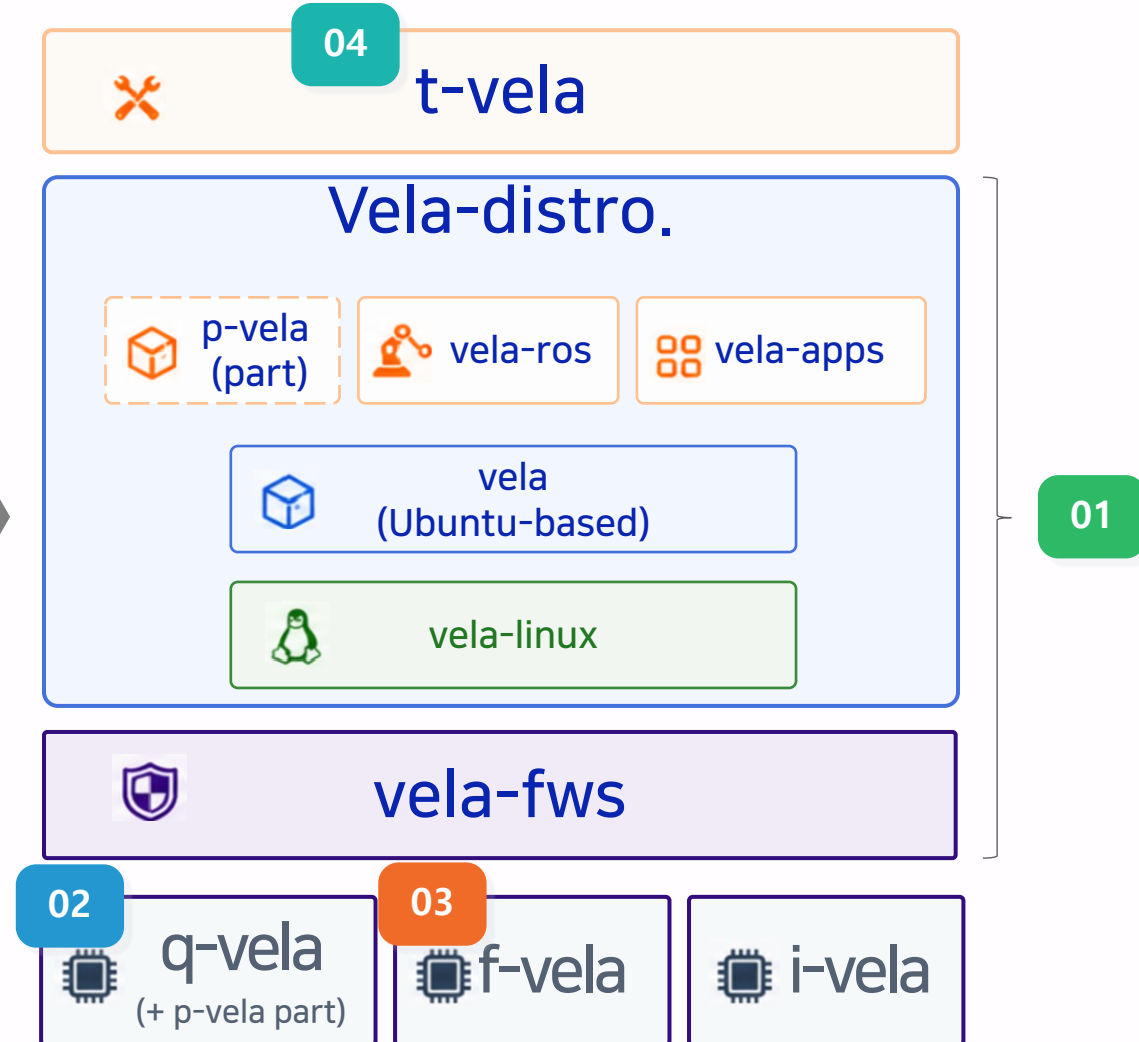
Repository	Role	Layer	Language	Status
vela-apps	I/F, Runtime/SDK	System SW	C/C++	Public
vela-ros	ROS2	System SW	C	Public
vela	Linux Distribution	OS	Shell	Public
vela-linux	Linux Kernel + Driver	OS	C	Public
p-vela	Power Management	System SW	Python	Planned
t-vela	Cross AI Compiler	Tools	C++	Public
vela-fws	Bootloader/Monitor	FW	C	Public
q-vela	RISC-V SW Emulation-QEMU	HW	C	Public
f-vela	RISC-V FPGA	HW	Chisel, Scala, C++	Public
i-vela	Enhanced RISC-V FPGA	HW	Chisel, Scala, C++	Planned

03

Relationships of Repositories & Software Distribution



RISC-V Vela Repositories



Riscv-vela Distro. (plan)

03 Vela

Ubuntu 24.04(Noble) 기반 베이스 배포판(Vela-OS)

① 저장소 구조

vela/

README.md 목표·검증환경·릴리스 정책

init.sh Ubuntu Noble RISC-V
부팅 이미지 생성 스크립트

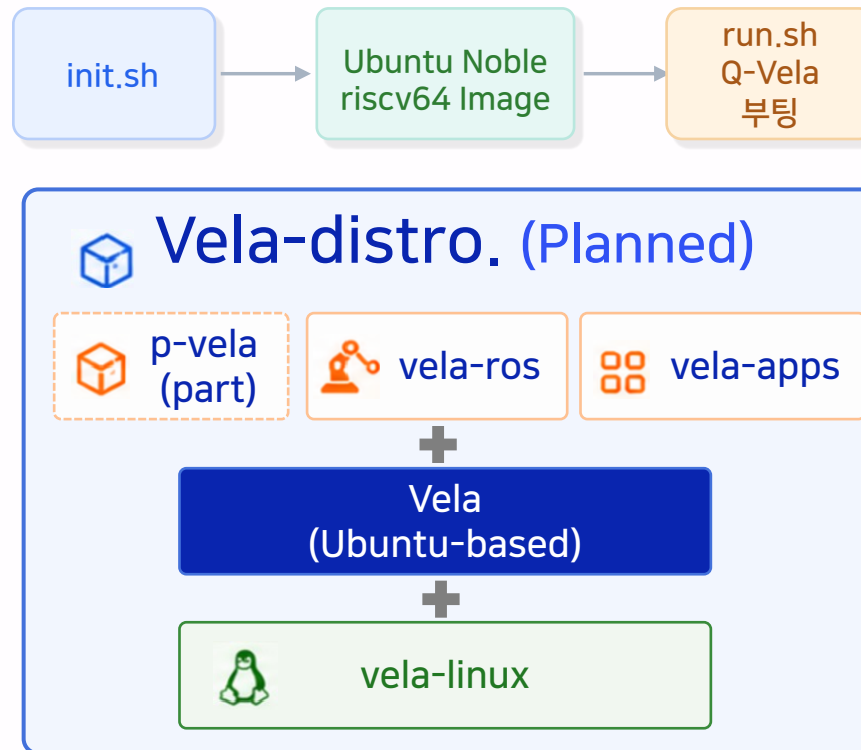
run.sh QEMU(Q-Vela) 부팅 실행

extlinux.conf U-Boot/extlinux 부팅 설정

netcfg.yaml Netplan 네트워크 설정

.github Issue template / 협업 기반

② 통합 구조



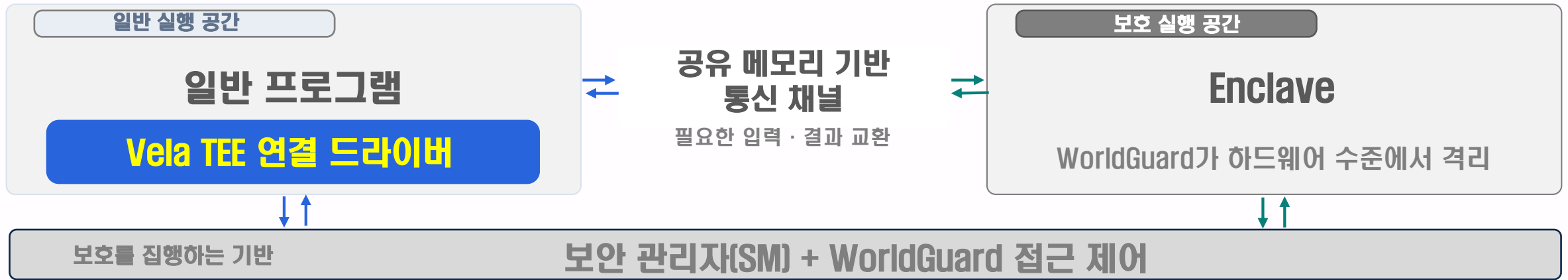
③ 특징과 장점

- Ubuntu 기반: **Debian 패키지 생태계와 호환** (apt 환경 제공)
- QEMU에서 빠르게 부팅·검증 후 FPGA/보드로 확장
- ROS2·DDS·AI Runtime을 묶는 상위 배포판 역할
- 개방형 RISC-V 시스템 SW 생태계의 기준 이미지로 활용 가능



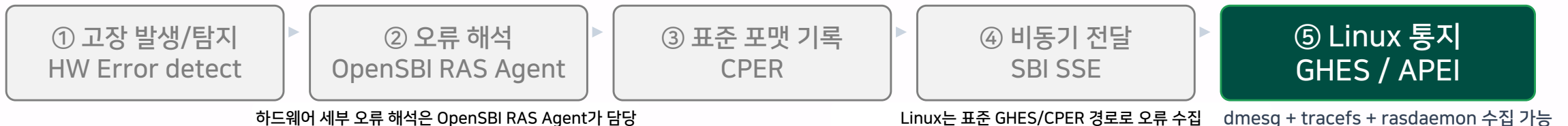
Vela-linux 고신뢰, 고가용 기능 확장

(고신뢰) Host-to-Enclave 연결 드라이버 추가



(고가용) RERI 표준 고장정보를 Linux 커널이 인지하도록 SW스택 수정

Linux 표준 오류 보고 체계





Vela-ROS

- ROS2 Jazzy Supported Platforms : x86-64 & ARM,

No RISC-V



Architecture	Ubuntu Noble (24.04)	Windows 10 (VS2019)	RHEL 9
amd64	Tier 1 [d][a][s]	Tier 1 [a][s]	Tier 2 [d][a][s]
arm64	Tier 1 [d][a][s]		
arm32	Tier 3 [s]		

- Vela-ROS : RISC-V Ubuntu 환경에서 ROS 2 Jazzy 및 관련 패키지의 소스 빌드와 설치를 연결하는 자동화 도구

(의미) 단순 재컴파일이 아니라, 수백 개 패키지의 의존성을 해결하고 RISC-V에서 동작하지 않는 핵심 SW는 소스까지 직접 수정·포팅



Vela-ROS 패키지 & 설치 방법

- ROS 2 Jazzy Core + Nav2·SLAM·DDS 포함 총 425개 Packages

<독립 Vela-ROS APT 저장소 제공>

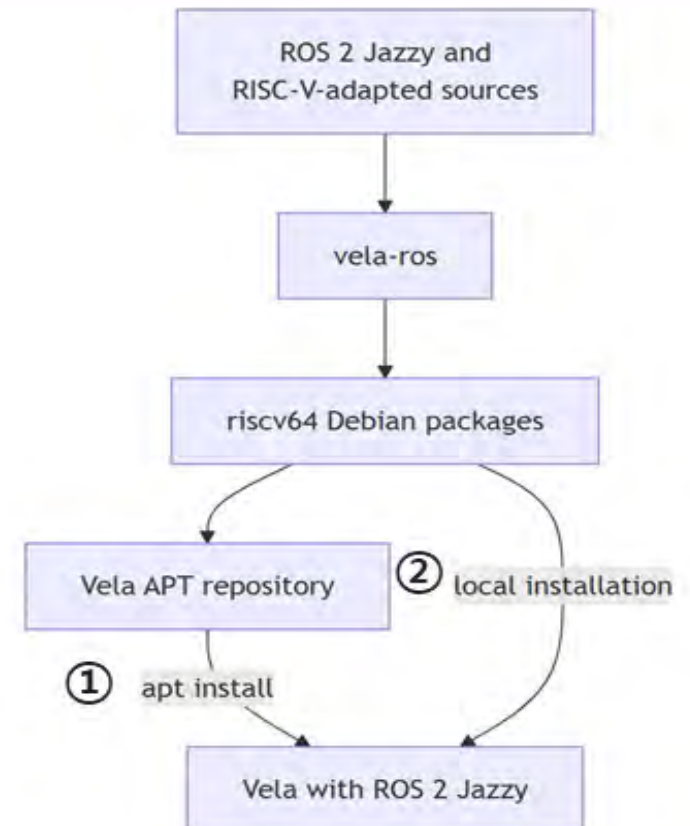
Vela-ROS package list

Repository: <http://vela.falinux.com/ubuntu/>

Latest packages: 425

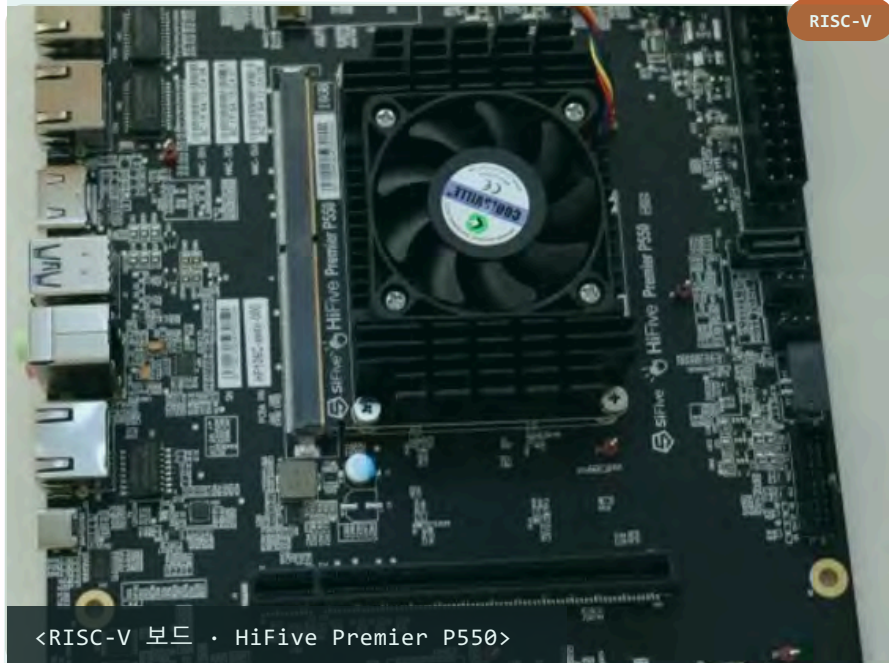
Package	Version	Architecture	Download
python3-bloom	0.13.0-101	all	python3-bloom_0.13.0-101_all.deb
python3-catkin-pkg	1.1.0-101	all	python3-catkin-pkg_1.1.0-101_all.deb
python3-catkin-pkg-modules	1.1.0-2	all	python3-catkin-pkg-modules_1.1.0-2_all.deb
python3-colcon-alias	0.1.1-100	all	python3-colcon-alias_0.1.1-100_all.deb
python3-colcon-argcomplete	0.3.3+upstream-1	all	python3-colcon-argcomplete_0.3.3+upstream-1_all.deb
python3-colcon-bash	0.5.0-100	all	python3-colcon-bash_0.5.0-100_all.deb
python3-colcon-bazel	0.1.0-100	all	python3-colcon-bazel_0.1.0-100_all.deb
python3-colcon-cd	0.2.1-100	all	python3-colcon-cd_0.2.1-100_all.deb
python3-colcon-clean	0.2.1-100	all	python3-colcon-clean_0.2.1-100_all.deb
python3-colcon-cmake	0.2.29-100	all	python3-colcon-cmake_0.2.29-100_all.deb
python3-colcon-common-extensions	0.3.0-100	all	python3-colcon-common-extensions_0.3.0-100_all.deb
python3-colcon-core	0.20.1+upstream-1	all	python3-colcon-core_0.20.1+upstream-1_all.deb
python3-colcon-coveragepy-result	0.0.8-100	all	python3-colcon-coveragepy-result_0.0.8-100_all.deb
python3-colcon-defaults	0.2.9+upstream-1	all	python3-colcon-defaults_0.2.9+upstream-1_all.deb

<설치방법>



03

Vela-ROS/TurtleBot4 ROS2 응용 데모 환경

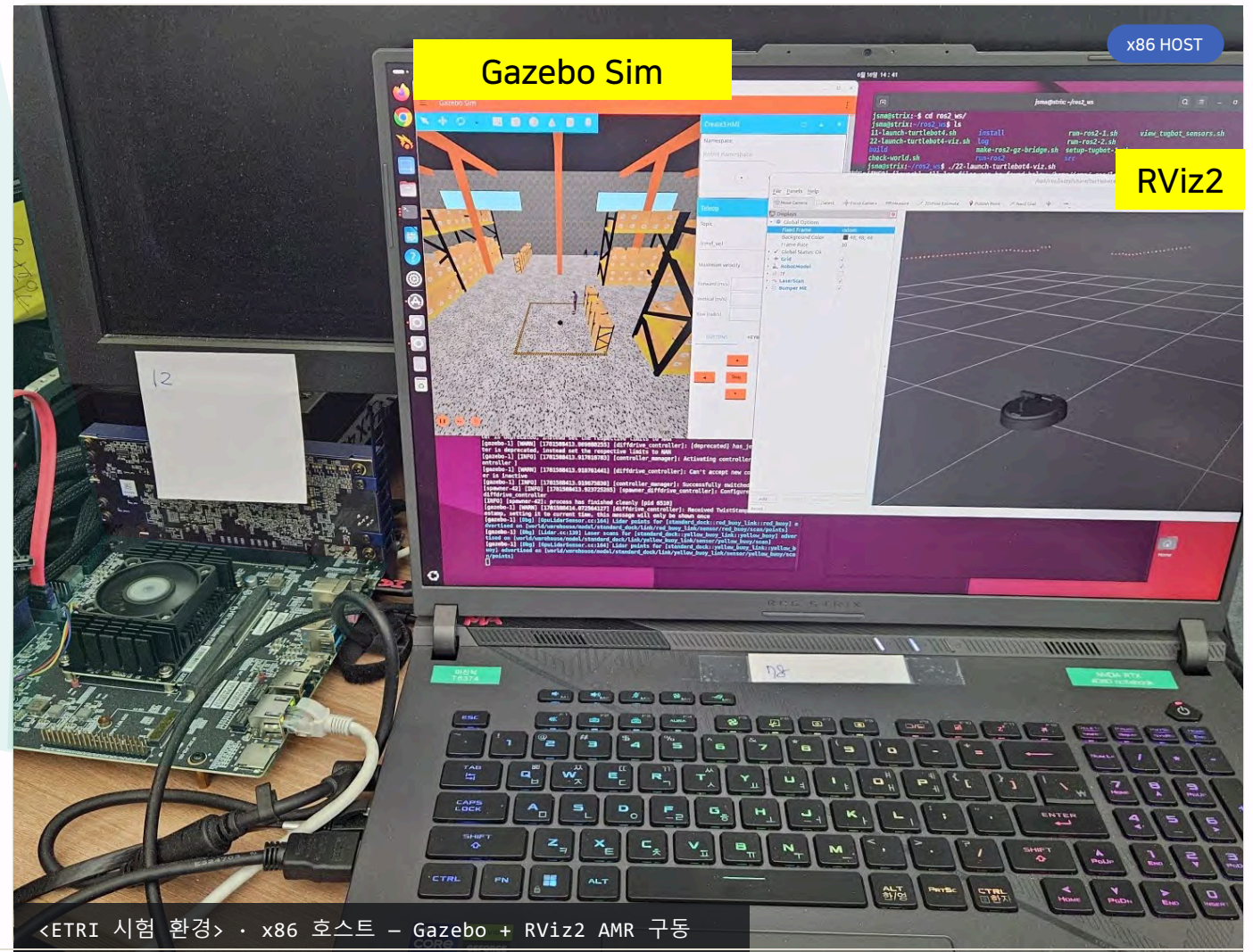


SLAM, NAV2 응용 구동

Vela-ROS (ROS2 Jazzy)

VELA (Ubuntu 24.04 riscv64)

SiFive HiFive Premier P550 보드



03

TurtleBot4 ROS2 응용 분산 실행 데모

The screenshot displays a ROS2 environment with three main components:

- Gazebo Sim:** A 3D simulation of a warehouse environment with yellow shelving units and a small robot. The interface includes a top toolbar and a bottom status bar.
- RViz:** A 2D visualization window showing a top-down view of the robot's position and the environment's geometry. It includes a menu bar (File, Panels, Help) and a toolbar.
- Terminal:** A command-line interface showing the execution of ROS2 commands and the output of the simulation. The output includes messages about ignoring velocities and the robot's position.

The terminal output shows the following commands and responses:

```

=====
lock bridge configuration
-----
lock_raw : Gazebo original clock
lock      : throttled ROS clock
bridge PID: 9581
turtle PID: 9639

: Ignoring velocities commanded wh
: Ignoring velocities commanded wh
: Ignoring velocities commanded wh

```

03

고신뢰기능 - 신뢰실행환경(TEE)

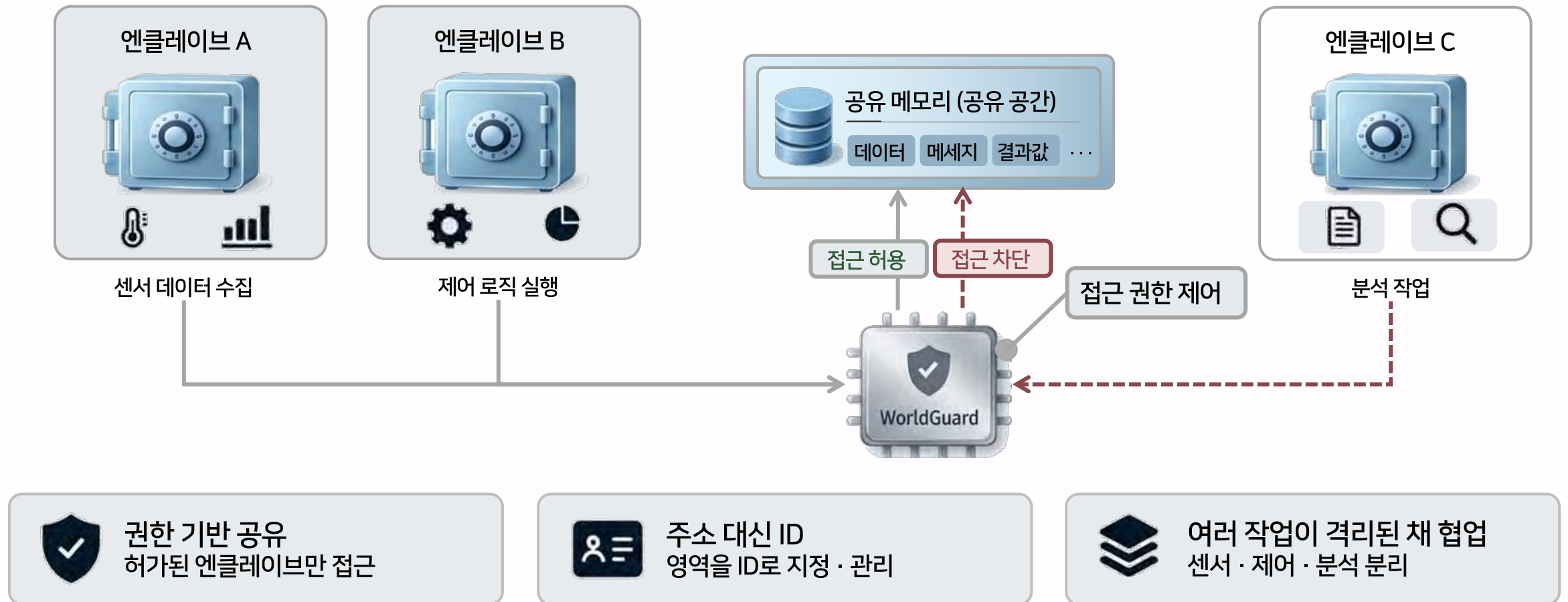
실행중 보호가 필요한 응용을 격리된 공간안에서만 실행하는 기술



03

엔클레이브간 데이터 공유 기능

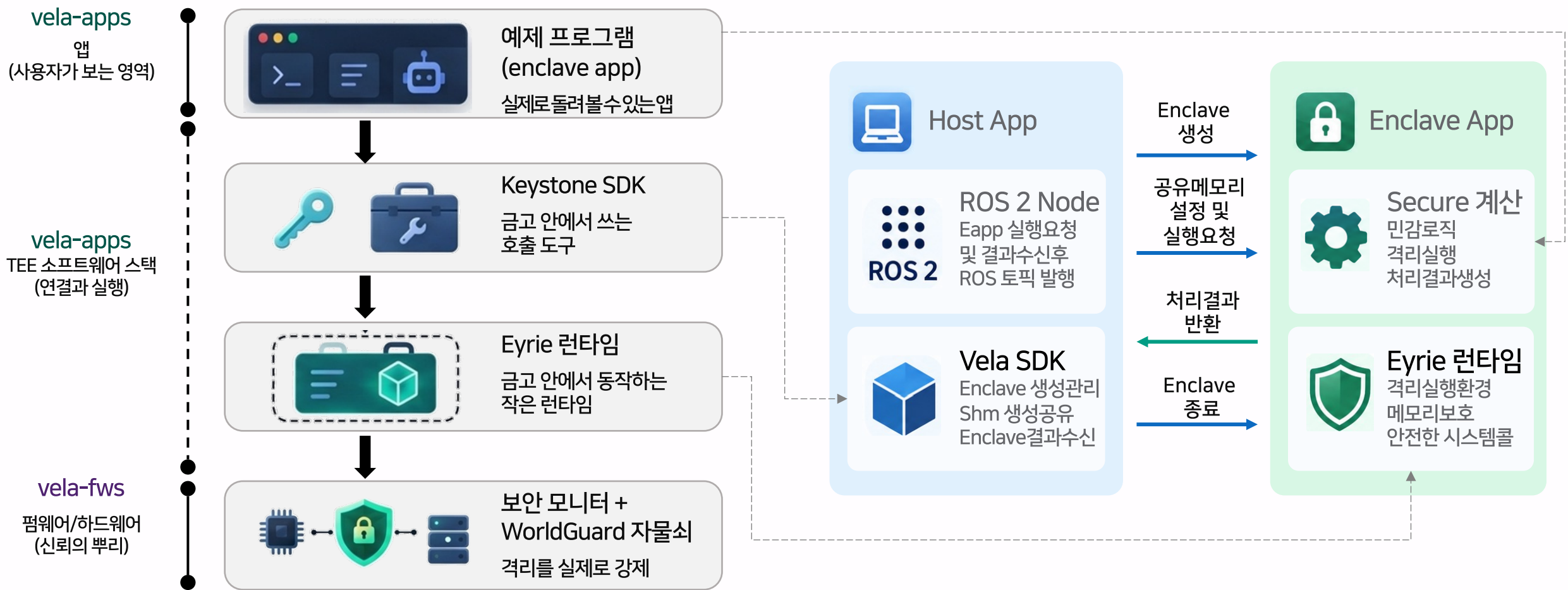
여러 금고가 안전하게 데이터를 나눔



03

Host-Enclave App 연동 구조

앱 실행이 하드웨어 격리까지 이어지는 경로 보장

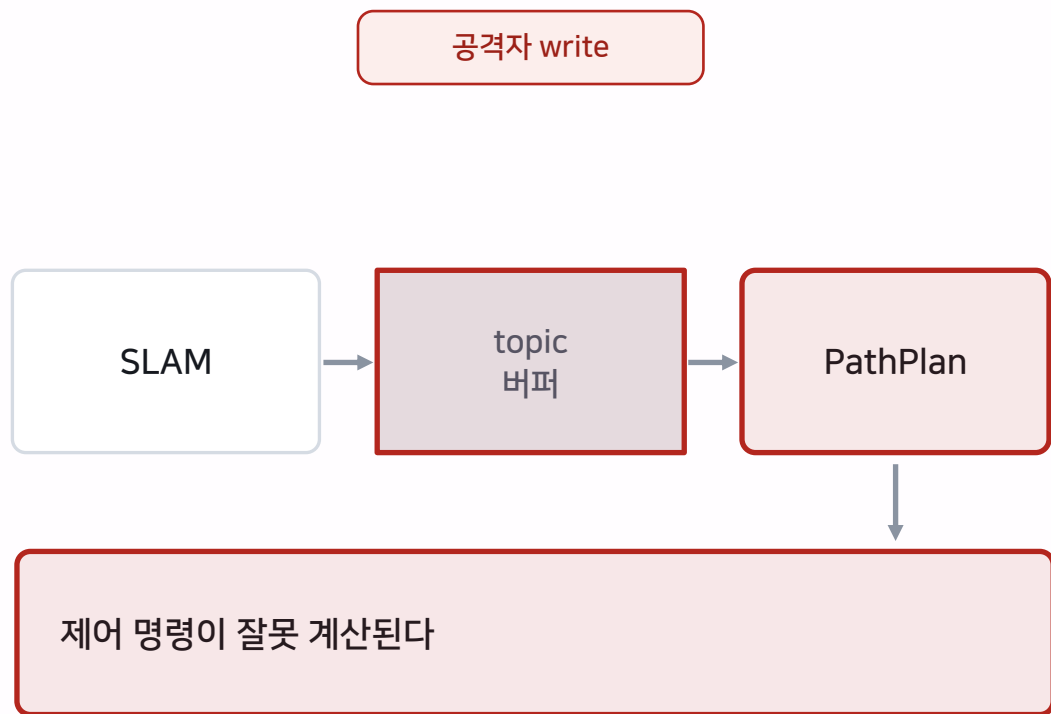


03

ROS 응용의 일반 실행 vs. 신뢰실행 비교

일반 실행 환경

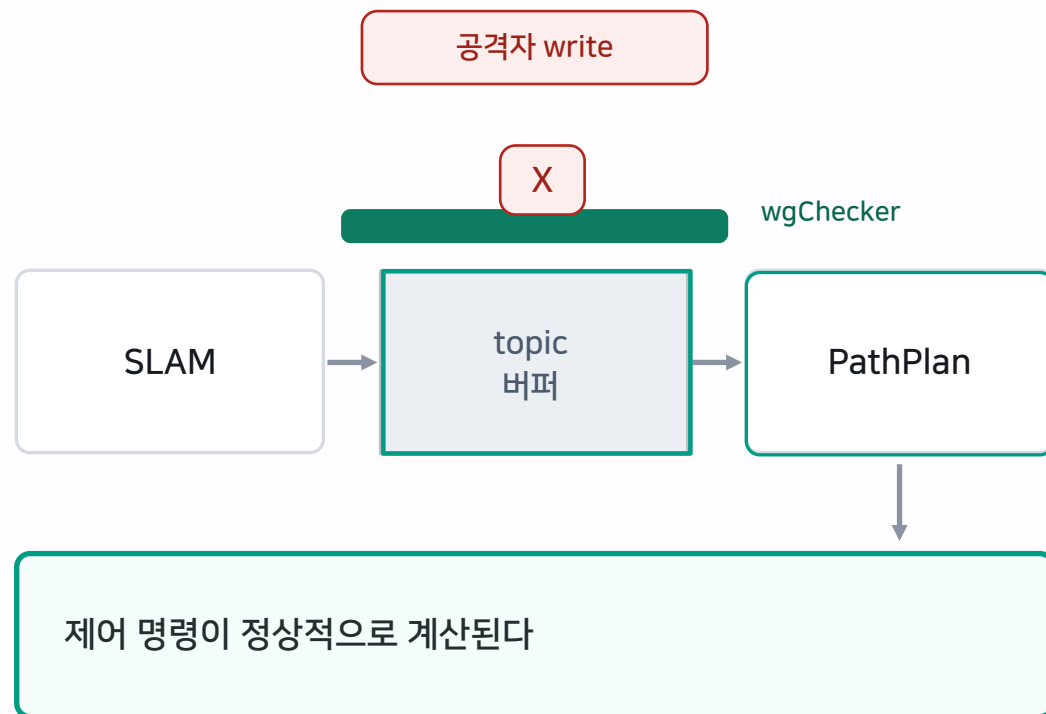
topic 버퍼가 host 메모리에 있다



로봇이 엉뚱한 경로로 주행한다

신뢰 실행 환경(Vela-TEE 적용)

topic 버퍼가 enclave 보호 영역에 있다

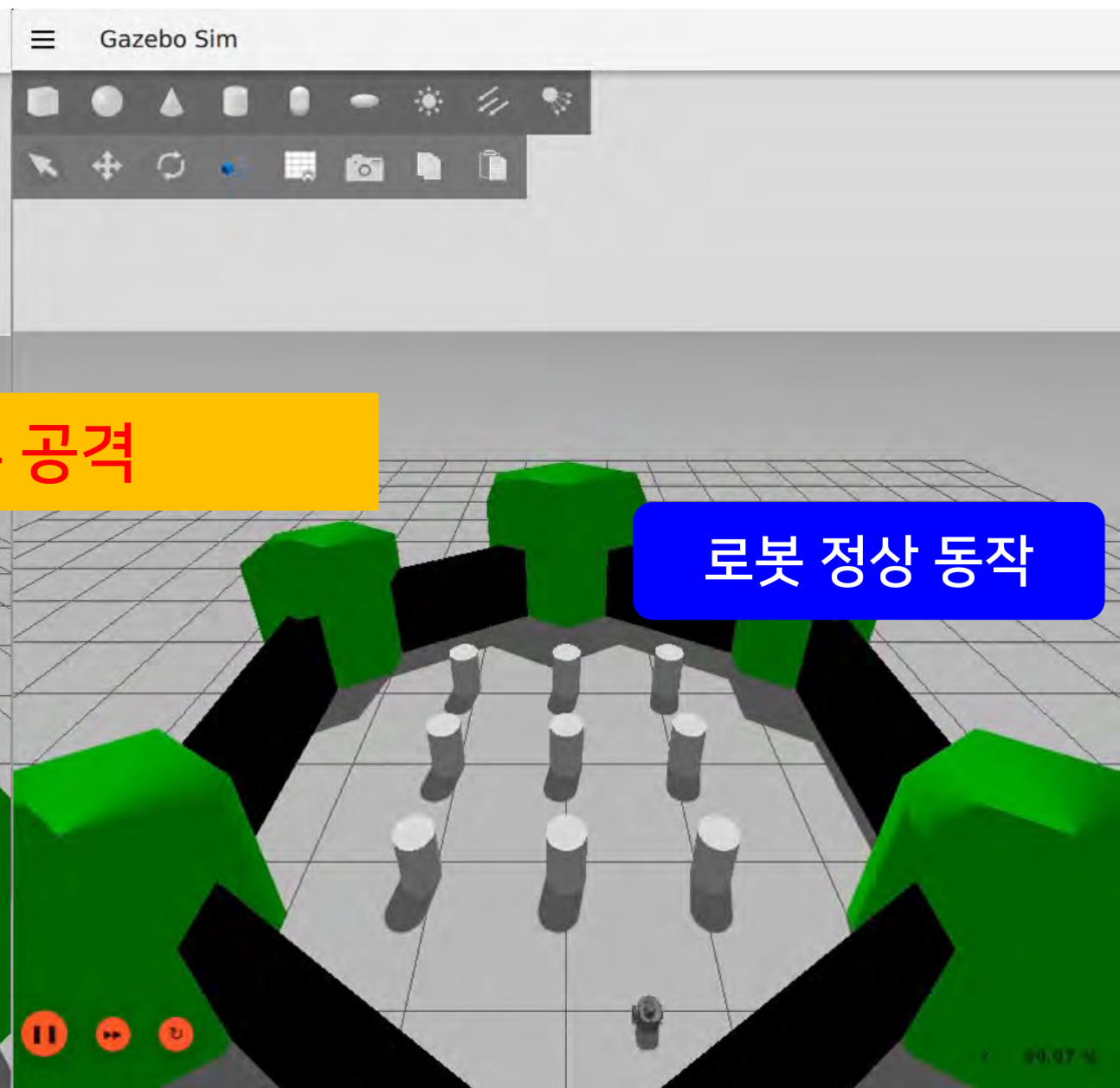


공격이 있어도 주행이 정상 유지된다

일반 실행 환경



신뢰 실행 환경





고가용(RAS) 기능

RAS는 고장 사건을 놓치지 않고 기록으로 남기는 기술

자동차의 블랙박스처럼



예기치 않은 사고도
기록이 남아야
원인 파악 가능



조용한 고장도 기록

사용자가 모르는
작은 오류도 기록



원인 추적 가능

언제, 어디서, 어떤
오류였는지 확인 가능

01 오류 발생



02 운영체제가 감지



03 기록 수집



04 가독성 있게 변환





고가용 RAS SW 전체 경로 검증

RAS SW 개발을 위해 시험용 오류를 주입해 전체 경로를 검증

01

주입

고장 주입기



원하는 종류·횟수·간격으로
오류 발생

02

탐지

커널/운영체제



하드웨어 오류 탐지

03

수집

로그 수집



상세 정보를 로그 기록으로
저장

04

해독

오류 해독기



16진수 오류 코드 기록을 문
장으로 변환

vela-apps가 제공하는 도구



고장 주입기

다양한 오류를 안전하게 주입



설치 스크립트

한 번에 설치하고 설정



오류 해독기

16진수 코드를 사람이 읽는 문장으로

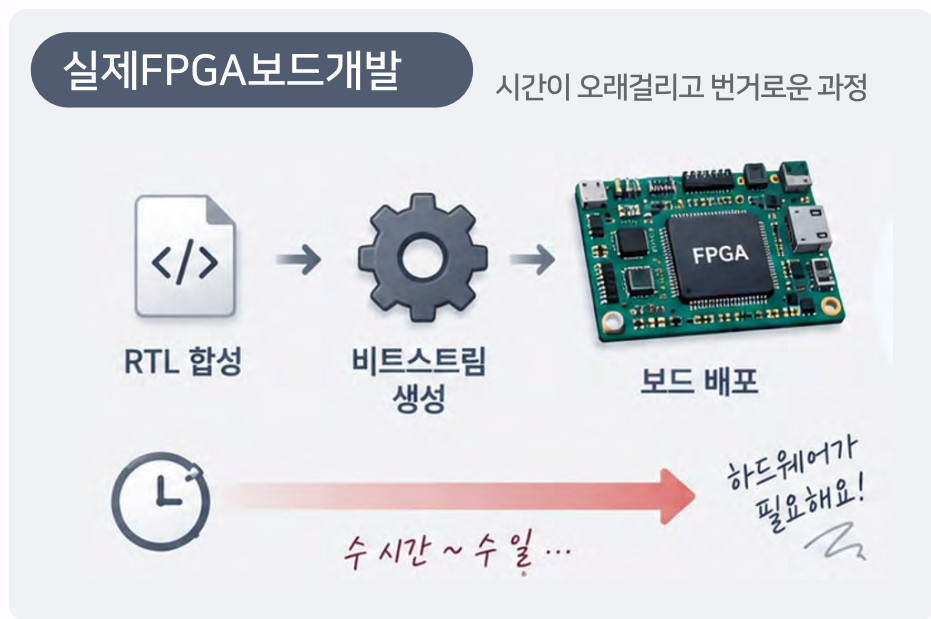


사용 문서

처음부터 활용까지 가이드

03 Q-Vela 개요

하드웨어가 나오기 전에 소프트웨어를 먼저 검증하는 에뮬레이터



보드가 없어도
커널·펌웨어·앱을
먼저 검증



개발 속도 향상



문제 조기 발견



비용 절감

03

Q-Vela 기반 시스템 SW 검증

가상 환경이지만 실제와 동일한 전체 부팅 흐름 제공



드라이버 기능/동작 검증

하드웨어와 소프트웨어의 인터페이스 확인



펌웨어 기능/동작 검증

부트로더부터 커널까지 정상 부팅 확인



앱 동작 검증

실제 환경과 유사한 실행 환경에서 동작 확인

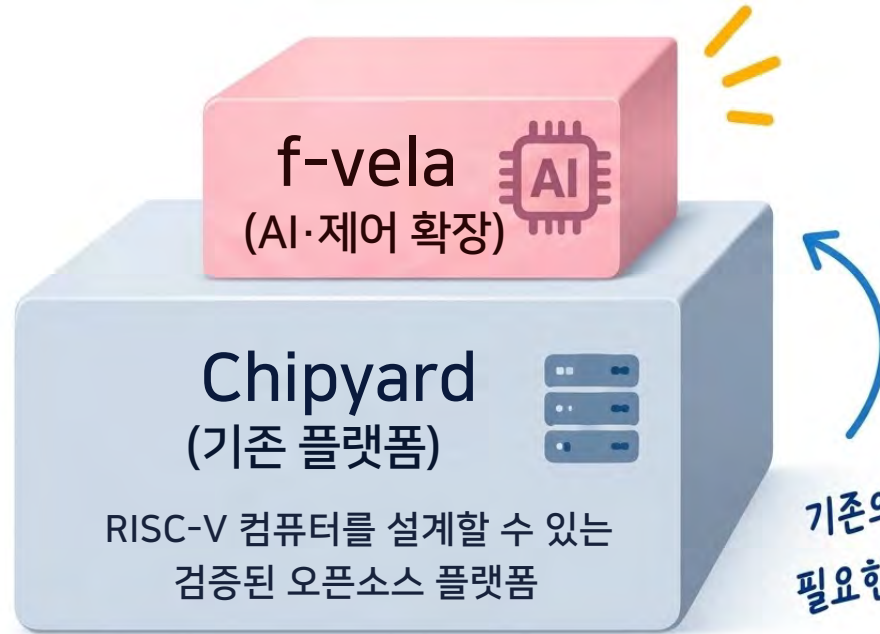
03

Q-Vela HW 에뮬레이션 기능



03

F-Vela 개요



f-vela는 AI·제어 가속기를
쉽게 실험하고 분석할 수 있게 만든
Chipyard 기반 HW 연구 플랫폼

기존의 장점은 그대로,
필요한 부분만 더 강하게!



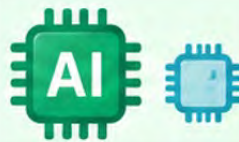
바꿔 보고,
실험하고,
더 나은 아이디어를
만들어가는 플랫폼!

01



검증된 설계 기반 재사용
버스, 캐시, 메모리 같은
기본 구조는 그대로 활용

02



AI·제어 연산 강화
NPU, VPU 같은
가속 기능을 추가

03



연구자가 직접 수정 가능
블랙박스가 아니라
바꿔 보고 실험할 수 있는 구조

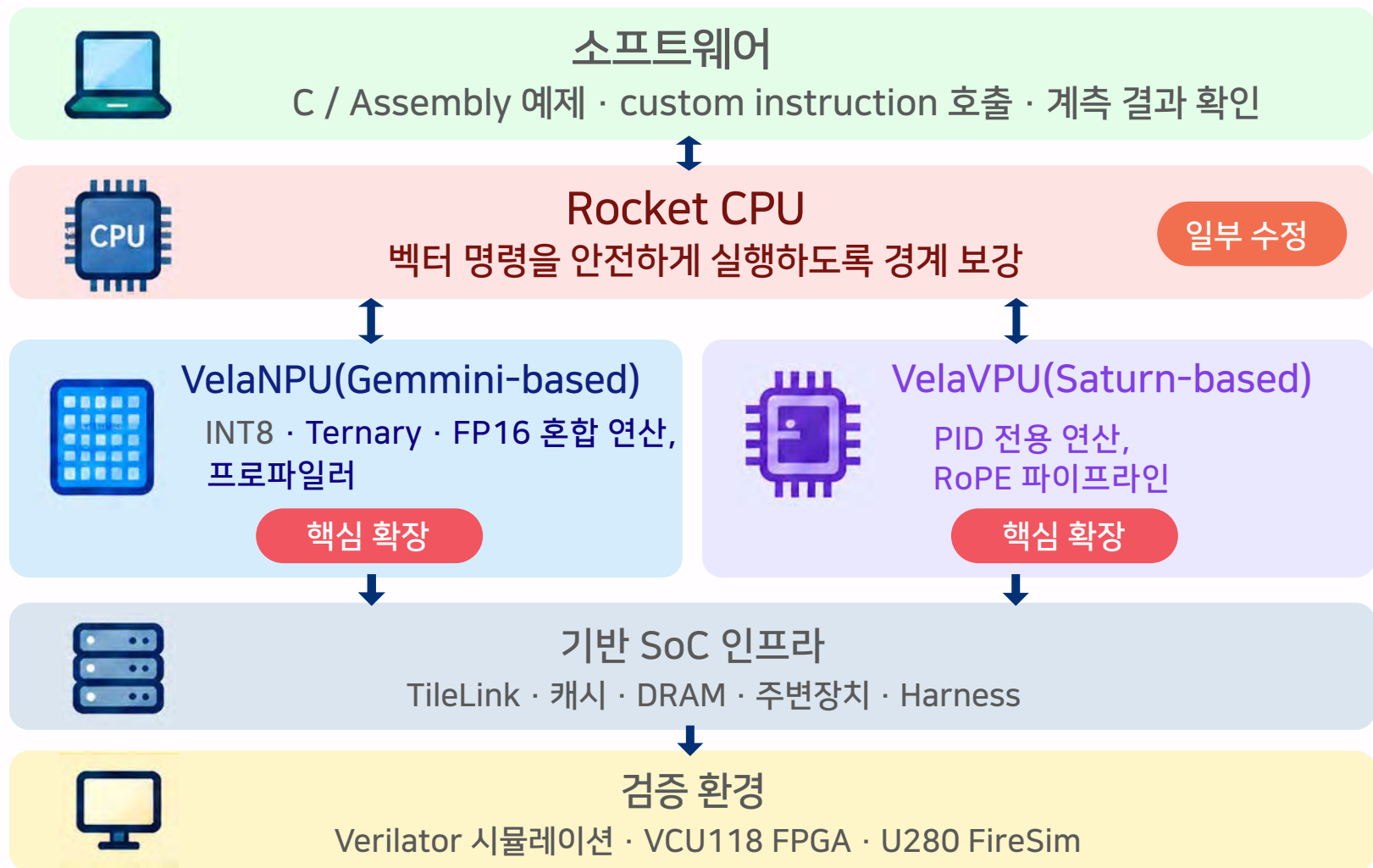
04



실행 원인까지 관찰 가능
왜 느린지, 어디가 병목인지
추적 가능

03

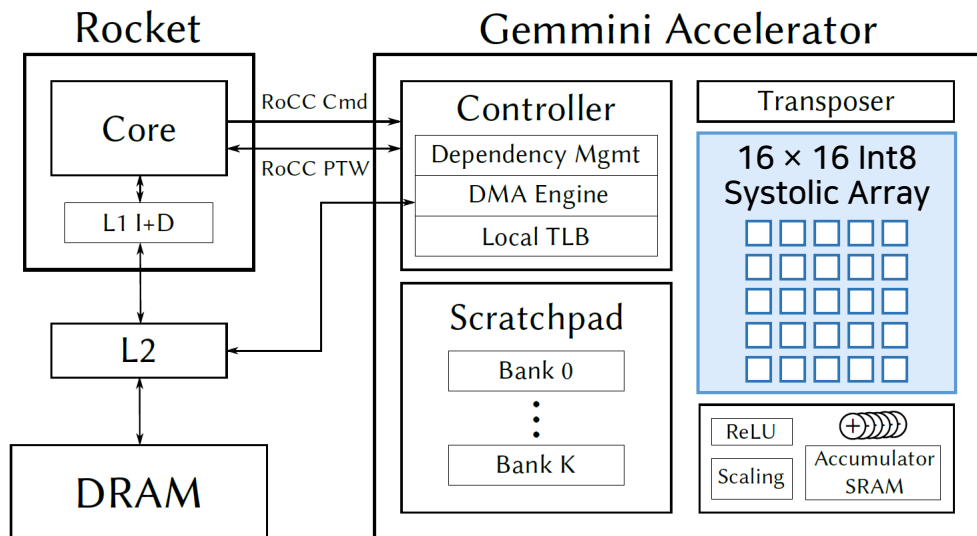
F-Vela 구조



03

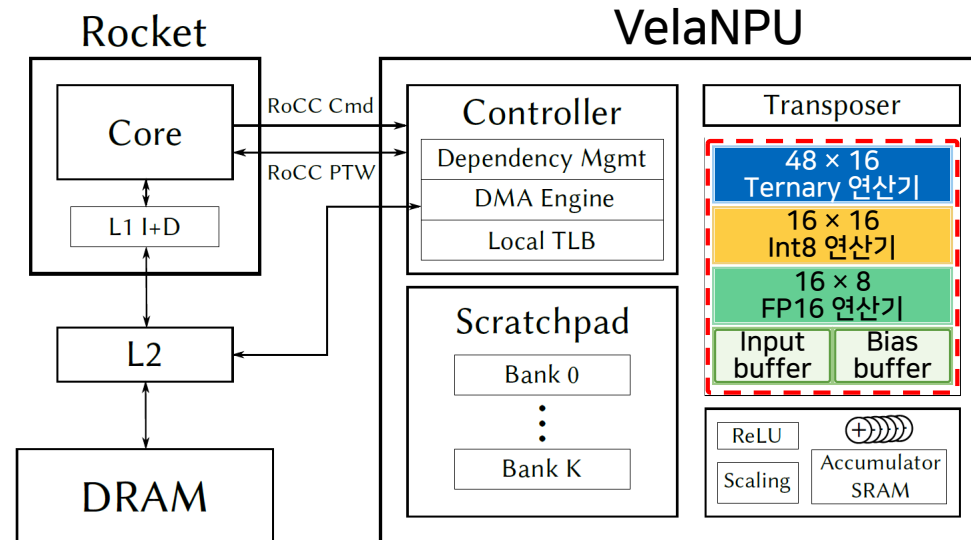
F-Vela : VelaNPU 핵심 기여

기존 구조: Gemini



단일 16×16 Int8 행렬 연산기 중심

변경 구조: VelaNPU



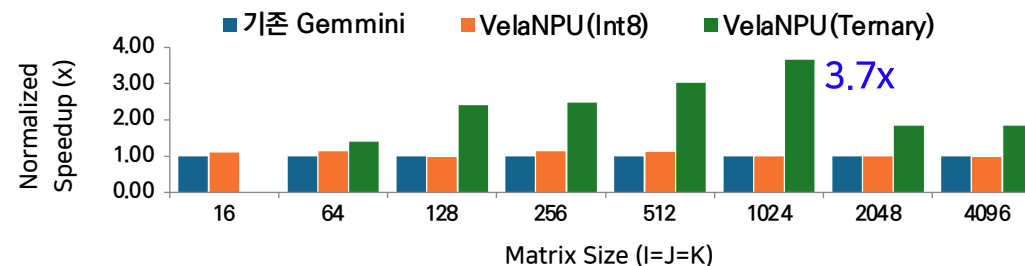
기존 16×16 INT8 연산기에 48×16 Ternary 및 16×8 FP16 연산기 추가

핵심 변화

무엇이 바뀌었나?

비교 항목	기존 Gemini	변경 VelaNPU
지원 연산	일반 Int8 행렬 연산	Int8 + Ternary(Int2×Int8) + FP16 연산 지원
데이터 경로	기본 Scratchpad 중심	Input/Bias buffer 추가, ternary·FP16 경로 확장
기대 효과	기본 행렬 가속	연산량 확대, 병렬성 증가, 특정 AI workload 성능 향상

GEMM 연산 성능 개선

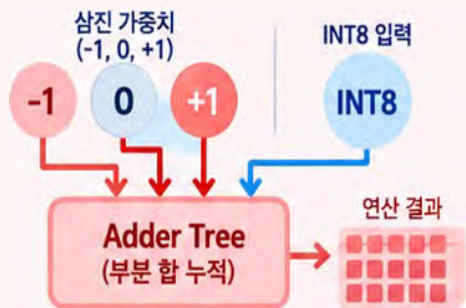


03

F-Vela 주요 특징

01 삼진(Ternary) 가중치 × INT8 혼합 연산

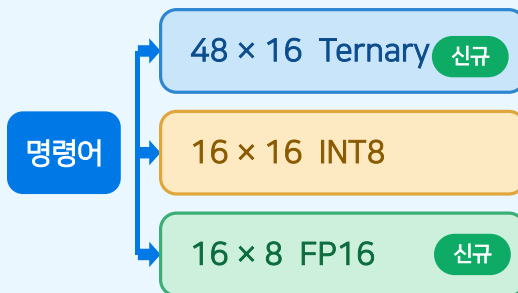
Systolic → Adder Tree
기반 하이브리드 연산기



! 가중치 저장 1/4
연산 효율을 확장

02 FP16 lane 추가와 정밀도 선택

정수 경로 옆에
부동소수점 경로를
나란히



! FP16 NPU처리,
CPU fallback 감소

03 왜 느린지 볼 수 있는 프로파일러

명령의 시작과 끝,
실제 메모리 응답까지 추적



! 병목 원인 분석

04 응용 특화 벡터 명령

실제 워크로드에 맞춘
벡터 명령 확장



! HW/SW 공동 설계 사례



F-Vela 데모 : BitNet 2B LLM Demo @ FPGA (×20배속)

```
Ubuntu x + v - □ ×  
# ./ask.sh "Explain what an NPU is in one sentence."  
█
```

03 T-Vela

PyTorch 모델에서 RISC-V AI 가속기까지, 비어 있던 길을 연결하는 컴파일러 툴체인



(문제) 추가된 커스텀 연산이 일반 연산으로 분해되어 의미 상실



F-Vela 커스텀 연산 추가 해법



표준 연산 → Linalg

범용성과 이식성을 확보



가속기 전용 연산
→ 전용 dialect

하드웨어의 의미와 성능을 보존

03 T-Vela 컴파일 흐름

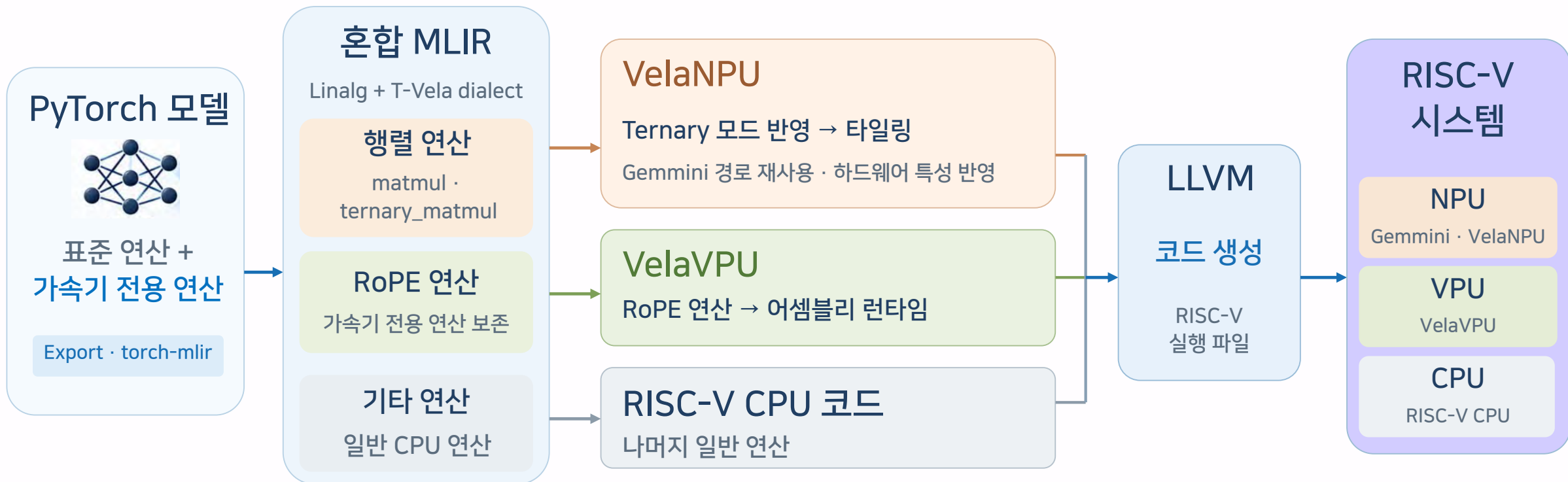
Torch-MLIR 입력을 받아, 연산에 맞는 실행 경로로 변환, 고효율 타겟 코드를 생성

01 AI 모델 정의

02 연산 의미 보존

03 연산 타깃별 변환

04 코드 생성 · 연결



* RISC-V 시스템: F-Vela Platform, Chipyard 기반 HW플랫폼(Gemmini)

03

T-Vela 컴파일 과정 및 커스텀 연산기 실행 성능

T-Vela가 PyTorch의 GEMM을 VelaNPU 실행 코드로 연결

01 Ternary OFF: INT8 가중치 및 INT8 출력

PyTorch 코드

```
class TorchGEMM(torch.nn.Module):
    def forward(self, x, w):
        return fvela_matmul(x, w, ternary="OFF")
```

02 Ternary ON: 2bit packed 가중치 및 INT8 출력

PyTorch 코드

```
class TernaryGemm(torch.nn.Module):
    def forward(self, x, w_packed):
        return fvela_matmul(x, w_packed, ternary='ON')
```

PyTorch 공통 함수 코드

```
def fvela_matmul(
    lhs: torch.Tensor, rhs: torch.Tensor, *,
    ternary: str = TERNARY_OFF) -> torch.Tensor:
    if ternary == TERNARY_ON:
        return ternary_matmul(lhs, rhs)
    return torch.matmul(lhs, rhs) ## TERNARY_OFF
```

MLIR 중간 결과

```
func.func @TorchGEMM(%arg0: memref<1x64xi8>, %arg1: memref<64x64xi8>
    %alloc = memref.alloca() : memref<1x64xi32>
    gemmini.tile_matmul %arg0 %arg1 %arg2 %alloc
    {noBias = true, ternary = false} :
    memref<1x64xi8> memref<64x64xi8> memref<1x64xi8> memref<1x64xi32>
    return
```

MLIR 중간 결과

```
func.func @TernaryGEMM(%arg0: memref<1x64xi8>, %arg1: memref<64x16xi8>
    %alloc = memref.alloca() : memref<1x64xi32>
    gemmini.tile_matmul %arg0 %arg1 %arg2 %alloc
    {noBias = true, ternary = true} :
    memref<1x64xi8> memref<64x16xi8> memref<1x64xi8> memref<1x64xi32>
    return
```

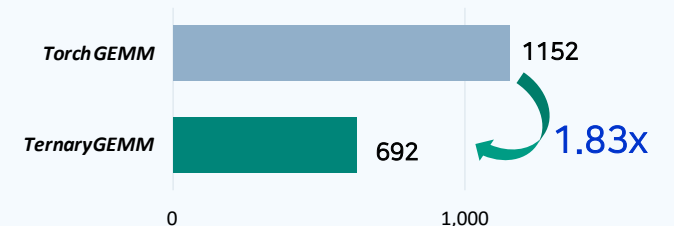
04 검증 환경

Cycle-accurate simulator (Verilator, Firesim with Alveo) 활용



03 실행 시간 결과

왼쪽의 Ternary ON/OFF 코드에서 생성한 ELF를 활용하여 측정
가로 축: 실행 시간 (Cycle)



T-Vela를 활용해 생성한 TernaryGEMM 실행 사이클이
TorchGEMM 대비 45.4% 감소



04

Release 및 라이선스 현황



Release 계획

01 릴리스 전략

정기 릴리스 주기

연 2회 정기 공식 릴리스 운영 ('26.08, '26.12)

통합 관리 방식

개별 저장소(Repository)의 개발 브랜치를 하나로 모아
통합 빌드 및 단일 검증 버전으로 배포

02 자동화 통합 CI 프로세스(진행중)





riscv-vela 저장소별 오픈소스 라이선스 현황

저장소	라이선스	오픈소스
vela	GPL-2.0 / Apache2.0 / BSD-3	Ubuntu 패키지
q-vela	GPL-2.0	QEMU
vela-linux	GPL-2.0	Linux kernel
t-vela	BSD-3 / Apache-2.0	LICENSE (UC Regents) + LICENSE.SiFive 병존 — Chipyard 계열
f-vela	Apach2.0 / BSD	NPU · Saturn · rocket-patch · vfrope 등
vela-apps	Apache-2.0 / MIT	sdk · runtime · ras · examples
vela-fws	BSD-2 / GPL-2.0	edk2 · opensbi · sm + buildroot
vela-ros	Apach2.0	ROS 2 Jazzy



(결론) RISC-V-Vela project 주요 기여

01

CPU ISA부터 응용까지 아우르는 최초의 RISC-V 기반 수직 통합 SDR 연구 플랫폼

CPU ISA 확장부터 NPU/VPU, 펌웨어, 보안(TEE), 신뢰성(RAS), Linux 커널 및 배포판, 그리고 ROS 2와 실사용 응용 레이어까지 통합 설계

02

조기 검증용 가상 HW 에뮬레이션 Platform인 Q-Vela로 빠른 프로토타입핑 지원

RISC-V QEMU Virt 머신에 NPU, Custom Vector ISA, WorldGuard TEE, RERI RAS HW를 통합.
RTL/FPGA없이 펌웨어·Linux·OS·고신뢰/고가용/AI 앱 개발 가능

03

최초로 425개의 riscv64용 ROS2 Debian 바이너리 패키지 제공으로 SDR 개발 촉진

독자적인 공개 저장소로 ROS2 Debian 패키지 제공, 복잡한 컴파일 수고 없이 APT로 빠르게 환경 구축이 가능

04

WorldGuard 기반 TEE와 RERI RAS 표준 기반 시스템 SW는 신뢰성/가용성 극대화

감사합니다

